

Now, notice that this is completely listener-based architecture, with the code kicking into action when we receive the data-set from the main script. Once that's done, we define the prerequisite common items that will be used throughout the code, such as the data-buckets to collect specific items from the transmitted object along with the visual entities for the page. We'll be structuring the whole thing as two donut charts (which are basically pie charts with a hole in them), so you can also see the values for their size. Also, there would need to be color scales as we can't afford to define the color structure for a data-set as dynamic as the one that we have here. We'll also apply a heading to the page, which we had extracted from the brandStringBundle used earlier in the main script.

Next, onto some D3 dirty work...

```

• <code>
•
•   var canvas = d3.select("#wrapper") // Create SVG on
'wrapper' div
•     .append("svg:svg")
•     .attr("width", width + margin.left + margin.right)
•     .attr("height", height + margin.top + margin.bottom);
•
•   var vis1 = canvas.data([memData]) // Create first visu-
allization, bind memData
•     .append("svg:g")
•     .attr("transform", "translate(" + width/4 + "," +
height/2 + ")");
•
•   var arc = d3.svg.arc() //this will create <path> ele-
ments for us using arc data
•     .outerRadius(rad)
•     .innerRadius(innerRad);
•
•   var outline = d3.svg.arc()
•     .outerRadius(rad+3)
•     .innerRadius(rad);
•
•   var pie = d3.layout.pie() //this will create arc data
for us given a list of values
•     .value(function(d) { return d.amount; });
•
• </code>

```